Research Statement - Marcelo d'Amorim

I develop **practical techniques that make software more reliable**, working at the intersection of **Software Engineering and Programming Languages**. Software bugs are costly and increasingly unavoidable, as ever more code is written by humans and synthesized by machine learning (ML). Through program analysis and systematic testing, my methods span the software quality lifecycle: bug prevention, detection, diagnosis, and repair.

Trajectory. I earned my PhD at the University of Illinois at Urbana-Champaign in 2007 and spent the following fifteen years building a research program in Brazil, where I was promoted to Associate Professor at the Federal University of Pernambuco in 2016. I joined NC State as a *tenured* Associate Professor in November 2022. This move was a deliberate step change rather than a continuation of the same trajectory: I rebuilt my group from the ground up (now six PhD students,[1] with all three of my PhD graduates already in faculty positions in the United States and Brazil), shifted the center of gravity of my funding to U.S. federal (NSF) and domestic industry sources, and reoriented my agenda toward AI. My master's alumni have likewise moved on to doctoral programs at TU Delft and Stuttgart and to engineering roles at companies such as AWS. My group has moved from making established analyses practical to *defining new testing methodologies* for software that resists precise specification: emerging AI-centric systems (Axis 1), where I place my greatest emphasis, and conventional software with partial specifications (Axis 2). Accordingly, this statement foregrounds work from the post-2022 period, situating it atop the foundation I built earlier.

Funding. My research has been supported by diversified funding totaling $1.78M over my career, including three grants from the U.S. National Science Foundation (NSF), an industry grant from L3Harris, and three grants awarded through international competition. The largest of the NSF grants, on which I am the sole PI, is currently active and funds my program on testing autonomous, AI-enabled systems (Axis 1). Since joining NC State in November 2022, my personal share of awards totals $956K.

Impact and Service. My group releases open-source tools we maintain for the research we do,[2] and they have demonstrable impact: they have revealed hundreds of bugs in widely-used systems, including C/C++ compilers (e.g., GCC and Clang) [6] and deep learning libraries (e.g., PyTorch and TensorFlow) [2,5], as well as security vulnerabilities in real-world code [44]. My publication record comprises 52 full conference papers and 22 journal articles, including many at the field's most selective venues (e.g., ICSE, ASE, FSE, ISSTA, PLDI, and IJCAI). This work has been recognized with a best-paper award at ICST 2026, a Microsoft SEIF award, and Distinguished Reviewer awards at FSE and ASE. I also serve the field in leadership roles: on the program committees of its most prestigious conferences, on the ESEC/FSE Steering Committee, as a guest editor for STVR, and as General Chair of FSE 2024,[3].

VISION

My vision is that research earns lasting impact by being *practical*: it pursues fundamental questions in the service of real-world use, what Stokes termed *use-inspired basic research*, or "Pasteur's quadrant."[4] Many powerful analyses, such as symbolic execution, runtime verification, and model checking, are precise in principle yet too costly, too noisy, or too hard to specify to be adopted in practice. I design lightweight techniques that retain the essential guarantees of these analyses while fitting everyday development workflows, scaling to real-world software, and producing results developers can act on. I hold my work to the standard of adoption: my constraint solver `CORAL` [62] was integrated into NASA's Symbolic PathFinder, and other researchers have independently built on my group's work in configurable-software testing [61] and test flakiness [40].

Rigorous experimental evaluation underpins my work. I treat a technique as only as credible as the evidence behind it, and I therefore assess my methods through controlled experiments on real-world software, following established empirical research methods[5] so that results are measurable, reproducible, and generalizable. In keeping with open-science standards, my group releases every project as open source, so others can independently reproduce and build on our findings.

The stakes are rising: as AI systems increasingly both write software and are themselves software, quality assurance must scale to code and models that no human fully understands. I intend for my group to help lead this shift, extending my agenda for practical, empirically grounded quality assurance to AI-generated code and AI-based systems.

ONGOING AND FUTURE WORK

My ongoing and future work pursues a single goal along two axes: making rigorous quality assurance possible where trustworthy *oracles*, the arbiters of whether software behaves correctly, are scarce or missing altogether. The first axis confronts this scarcity in its most acute form, the emerging AI-centric software that increasingly runs the world yet admits no precise specification; this is where I see the largest opportunity and place my greatest emphasis. The second carries the same philosophy to conventional software, turning the partial specifications developers already leave behind into practical assurance. My earlier work in runtime verification, regression testing, and developer-productivity tools underpins both and is summarized under Past Work.

[1] Students: `https://damorim.github.io/students.html`

[2] `https://github.com/ncsu-swat`

[3] Service: `https://damorim.github.io/service.html`

[4] D. E. Stokes. *Pasteur's Quadrant: Basic Science and Technological Innovation.* Brookings Institution Press, 1997.

[5] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering.* Springer, 2012.

Axis 1: Testing Emerging, AI-Centric Software

AI now underpins critical infrastructure, yet it is, in the end, software, and like all software it is riddled with bugs. What makes it uniquely hard to test is that correctness is frequently *undefined*: a deep-learning library, an autonomous vehicle, or a code-generating model rarely admits a precise specification or a reliable oracle, so even deciding whether a given run *failed* can be ambiguous. My group is therefore inventing testing methodologies purpose-built for AI-centric software, working outward along three fronts: the *infrastructure* AI is built on, the *systems* that embed AI in the physical world, and the *confidence* with which AI judges its own outputs.

★ *Testing AI infrastructure.* Every model in production inherits the correctness of the thin layer of deep-learning libraries beneath it, PyTorch, JAX, TensorFlow, so a single wrong operator there can silently corrupt results that no downstream test would catch. These libraries are nonetheless hard to test, because their operator APIs impose intricate, undocumented constraints on the shapes, types, and value ranges they accept, and most randomly generated inputs are rejected before any substantive code runs. My group learns these input specifications, symbolically and with ML, to synthesize inputs valid enough to execute deeply yet adversarial enough to expose bugs [2,8], complements this with coverage-guided fuzzing of library backends [5], and released a database of reproducible DL-library bugs to ground the community's work [14]. These tools have already revealed hundreds of bugs, and I plan to push them across frameworks and down into the compilers and GPU kernels beneath them.

★ *Testing AI-enabled systems.* When AI acts in the physical world, as in a self-driving vehicle, the space of situations it must handle is effectively infinite while the dangerous ones are vanishingly rare, so testing drawn uniformly from that space spends almost all of its effort on cases that were never going to fail. The key to efficiency is to let the system reveal where to look, since a model under stress leaves traces of its own uncertainty. My group turns those traces into test guidance: ThirdEye reads a model's attention maps to flag unsafe decisions [25], XMutant converts explainable-AI signals into targeted input mutations [10], and Foresee predicts where a system will misbehave and focuses generation there [4]. These signals come from the system under test; looking further ahead, I want to lift the same principle to the system's *environment* through *world models*, learned predictors of environment dynamics that would let us simulate the rare, safety-critical scenarios that matter most and anticipate failures before deployment, turning cyber-physical-systems testing from a reactive search into a prediction-driven methodology.

★ *Measuring the self-confidence of AI.* Beneath both fronts lies a single, deeper obstacle: AI is often *confidently wrong*, emitting plausible code, decisions, and repairs with no dependable internal signal of when it should be distrusted, so AI-generated code is frequently insecure or buggy [12,20] and even LLM-based repair remains unreliable [19]. If a system could faithfully report *how sure it is*, and that self-report were calibrated against how often it is actually right, we could concentrate scarce human review and testing effort exactly where the risk lies. I am beginning to study how to measure and calibrate this self-confidence, for AI models and for the code they generate, flagging low-confidence outputs for scrutiny. I see this as a foundation for the trustworthy deployment of AI and the natural culmination of my agenda on practical, empirically grounded quality assurance.

Axis 2: Practical Assurance Without Complete Specifications

Even conventional software, where correctness is in principle well understood, rarely ships with specifications that a tool can act on. But developers constantly produce *partial specifications* as a byproduct of their work: the assertions inside a unit test, the entry points a debloating tool is told to preserve, the trail of vulnerabilities an ecosystem has already suffered. My insight is that these fragments, though never written as formal specifications, are precise enough to power strong analyses once paired with the right tools, for example differential execution, coverage-guided fuzzing, and mutation. I recover and exploit them along three fronts.

★ *Lifting tests into self-generalizing oracles.* Automated testing rests on an awkward asymmetry: we can generate a million inputs automatically, but we cannot automatically judge whether any of their outputs is correct, the *oracle problem*. Often, though, that judgment is already written down and discarded: every unit test encodes a small, human-curated specification, in its assertions, expected exceptions, and pre/post-conditions, that we stop using the moment the test passes. My insight is that these fragments can be *generalized* rather than merely replayed: I mine them from existing tests, lift each from a single example into a parameterized predicate, using static analysis and LLMs, paired with a synthesized typed input generator, and fuzz the result under coverage guidance, turning a static test suite into a continuously widening bug-finding engine. The crux is *soundness*, since an over-generalized oracle raises false alarms; I address it by synthesizing an ensemble of independent implementations that conform to each predicate and letting their agreement arbitrate. This builds on my group's work on oracle generation [9,16], fuzzing class specifications [27], and executing partially specified code [18], and anchors *LASER*, a new collaborative project.

★ *Grading debloating specifications.* Debloating hardens software by stripping out unused functionality and shrinking its attack surface, but that safety rests entirely on a *debloating specification* of what to keep, which is almost always subtly wrong: too aggressive and it removes behavior the program still needs, too timid and it leaves exploitable code in place. Reportedly over a third of debloated binaries fail as a result,[6] yet developers cannot grade a specification before shipping it. My insight is that grading needs no formal specification, only the two programs themselves: I measure the behavioral divergence between the original and debloated versions as a two-sided signal, behavior wrongly dropped and behavior wrongly retained, populated through differential execution by a coverage-guided fuzzer steered toward divergence. That signal drives closed-loop refinement of the specification toward a concrete, checkable exit condition, and fits the CI/CD pipelines used for mission-critical software.

★ *Anticipating software-supply-chain risk.* Supply-chain security is stuck in a reactive posture: we flag a vulnerable dependency only after it is disclosed, always a step behind the attacker, and we learn little about *why* one ecosystem relays a weakness

[6] M. D. Brown et al. *A Broad Comparative Evaluation of Software Debloating Tools.* USENIX Security Symposium, 2024.

to thousands of downstream packages while another contains it. My insight borrows the central move of mutation testing, deliberately injecting faults to measure how well a system copes, and lifts it from a single test suite to an entire dependency ecosystem: I inject synthetic but realistic vulnerabilities into packages, calibrate them against historical CVEs, and trace how they propagate across transitive dependencies such as those in npm. Their propagation directly measures an ecosystem's *latent* exposure to vulnerabilities not yet discovered, and yields susceptibility maps, ground-truth propagation benchmarks, and predictive risk scores, extending my group's work on supply-chain and security measurement [15,20].

PAST WORK

This section summarizes the work I did, grouped by area. **Symbolic Execution and Constraint Solving [54,59,62,64, 67,69]**. Symbolic execution generates test inputs by deriving constraints for distinct program paths and solving them, but constraints from general programs often build on undecidable or intractable theories that off-the-shelf solvers cannot handle, limiting the paths a tool can explore. We developed the constraint solver `CORAL` [62,64,67,69], which combines interval constraint solving with search to tackle complex constraints (e.g., involving floating-point variables and math functions), and `qCORAL` [54, 59], a variant that quantifies solution spaces via Monte Carlo simulation to estimate the probability of program events. Both solvers are publicly available. **Program Model Checking [66,71–75].** State-space exploration is the essence of model checking, where time efficiency is central. Delta Execution (DE) [71–74] explores the search space with *sets* of concrete states, exploiting data similarity across states to speed up execution, backtracking, and state comparison; it improved exploration time up to 130x in the popular Java PathFinder model checker. Mixed Execution [66,75] uses a more efficient state representation for deterministic blocks, improving overall exploration time up to 37%. **GUI Testing [47,70].** In collaboration with Motorola, my team developed fuzzing techniques to find crashes and freezes in cell phones by exercising the UI of built-in apps [70], comparing strategies based on recombining existing UI-test fragments and on exploring GUI components exhaustively or randomly. More recently, with a team from Georgia Tech, I helped synthesize Android Espresso (UI) tests from natural language bug reports to ease bug reproduction [47]. **Runtime Verification (RV) [32,37,78,80–82].** RV monitors software properties at runtime and, beyond self-adaptation of safety-critical systems, has become practical during testing, revealing hundreds of bugs in code that violates Java API specifications. I explored efficient monitors [78,80,81], ML classifiers to prioritize the many (often false-positive) violations [23,37], and showed that developers tend to prefer imperative over declarative specifications [32]; recently we made monitoring faster and evolution-aware [3,13] and studied cryptographic-API misuse [21]. **Bug Finding in Configurable Software [36,46,50,55,61].** Bugs can hide in specific configurations because the configuration space is often too large to test exhaustively. SPLat [61] treats configuration choices as non-deterministic and uses symbolic execution to generate test inputs covering distinct configurations, finding bugs in complex systems such as the GCC compiler infrastructure [36,46,50, 55]. **Security Testing [28,44,56,65,68].** I proposed static information-flow analyses to identify vulnerabilities in PHP web applications (e.g., SQL injections) [56,65,68], a method to flag vulnerable IoT devices by analyzing their controlling mobile apps [44], and an algorithm to synthesize Network Intrusion Detector rules from malicious and benign traffic examples [28]. **Software Fault Localization [43,48,51,52,60,63].** Fault localization identifies the program statements likely responsible for a failure. I explored techniques to improve their precision and developers' debugging effectiveness, combining statistical fault localization with change-impact analysis [63] and incorporating human feedback [48,52]. **Test/Code Repair[31,45].** I explored repairing test cases [45] and Infrastructure-as-Code scripts such as Dockerfiles [31]. In both, the culprit is typically a change (e.g., an API update that breaks referring tests, or a "latest" version tag that forces a dependency upgrade) that automated refactorings cannot always absorb. **Regression Testing and Flaky Tests [17,24,29,33,39,40,49].** Flaky tests pass or fail nondeterministically. My group removed flakiness from test parallelism [29,49], detected it statically and by injecting noise [33, 39,40], and characterized it across languages and computing environments [17,24]. **Developer Productivity and Empirical Studies [1,7,15,22,26,42].** I built tools that ease everyday development, from assisting Q&A-forum users with Docker [42] and reusing API examples for Node.js [22] to linting Infrastructure-as-Code [26], and studied configuration, supply-chain, and portability defects in real projects [1,7,15].

PUBLICATIONS

[1] Yue Zhang, Uchswas Paul, **M. d'Amorim**, and Akond Rahman. Configuration Defects in Kubernetes. In *IEEE Transactions on Software Engineering (TSE)*. 2026.

[2] M. M. Abid Naziri, Shinhae Kim, Feiran (Alex) Qin, **M. d'Amorim**, and Saikat Dutta. Testing Deep Learning Libraries via Neurosymbolic Constraint Learning (Centaur). In *International Conference on Software Engineering (ICSE)*. 2026.

[3] Pengyue Jiang, Kevin Guan, Mahdi Khosravi, Moustafa Ismail, **M. d'Amorim**, and Owolabi Legunsen. Fine-Grained Analyses for Evolution-Aware Runtime Verification. In *International Conference on Software Engineering (ICSE)*. 2026.

[4] M. M. Abid Naziri, Stefano Carlo Lambertenghi, Andrea Stocco, and **M. d'Amorim**. Misbehavior Forecasting for Focused Autonomous Driving Systems Testing. In *International Conference on Software Engineering (ICSE)*. 2026.

[5] Feiran (Alex) Qin, M. M. Abid Naziri, Hengyu Ai, Saikat Dutta, and **M. d'Amorim**. How Effective Is Coverage-Guided Fuzzing to Test Deep Learning Library APIs? (FlashFuzz). In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 2026.

[6] Lingjun Liu, Feiran (Alex) Qin, Owolabi Legunsen, and **M. d'Amorim**. Learning Compiler Fuzzing Mutators from Historical Bugs. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2026.

[7] Denini Silva, MohamadAli Farahat, and **M. d'Amorim**. An Empirical Analysis of Portability Issues in Python Projects. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2026.

[8] Facundo Molina, M. M. Abid Naziri, Feiran (Alex) Qin, Alessandra Gorla, and **M. d'Amorim**. Improving Deep Learning Library Testing with Machine Learning. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. 2026.

[9] Facundo Molina, Alessandra Gorla, and **M. d'Amorim**. Test Oracle Automation in the Era of LLMs. In *ACM Transactions on Software Engineering and Methodology (TOSEM)*. 2025.

[10] Xingcheng Chen, Matteo Biagiola, Vincenzo Riccio, **M. d'Amorim**, and Andrea Stocco. XMutant: XAI-based Fuzzing for Deep Learning Systems. In *Empirical Software Engineering (EMSE)*. 2025.

[11] R. Melo, Pedro Simões, Rohit Gheyi, **M. d'Amorim**, Márcio Ribeiro, Gustavo Soares, Eduardo Almeida, and Elvys Soares. Agentic LMs: Hunting Down Test Smells. In *IEEE Software*. 2025.

[12] Liliana Pasquale, Antonino Sabetta, **M. d'Amorim**, Péter Hegedűs, Mehdi Tarrit Mirakhorli, Hamed Okhravi, Mathias Payer, Awais Rashid, Joanna C. S. Santos, Jonathan M. Spring, Lin Tan, and Katja Tuma. Challenges to Using Large Language Models in Code Generation and Repair. In *IEEE Security & Privacy*. 2025.

[13] Kevin Guan, **M. d'Amorim**, and Owolabi Legunsen. Faster Explicit-Trace Monitoring-Oriented Programming for Runtime Verification. In *ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 2025.

[14] M. M. Abid Naziri, Aman Kumar Singh, Benjamin Wu, Feiran (Alex) Qin, Saikat Dutta, and **M. d'Amorim**. BugsInDLLs: A Database of Reproducible Bugs in Deep Learning Libraries. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2025.

[15] Sarah Elder, Alex Klevan, Ranindya Paramitha, **M. d'Amorim**, and Laurie Williams. Aggregating Security Measures from the Dependency Tree. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED)*. 2025.

[16] Ishrak Hayet, Andrew Scott, and **M. d'Amorim**. ChatAssert: LLM-based Test Oracle Generation with External Tools Assistance. In *IEEE Transactions on Software Engineering (TSE)*. 2024.

[17] Denini Silva, Martin Gruber, Satyajit Gokhale, Ellen Arteca, Alexi Turcotte, **M. d'Amorim**, Wing Lam, Stefan Winter, and Jonathan Bell. The Effects of Computational Resources on Flaky Tests. In *IEEE Transactions on Software Engineering (TSE)*. 2024.

[18] Ishrak Hayet, Andrew Scott, and **M. d'Amorim**. Feedback-Directed Partial Execution. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024.

[19] U. Kulsum, Haotian Zhu, Bowen Xu, and **M. d'Amorim**. A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback. In *ACM International Conference on AI-Powered Software (AIware)*. 2024.

[20] Sivana Hamer, **M. d'Amorim**, and Laurie Williams. Just Another Copy and Paste? Comparing the Security Vulnerabilities of ChatGPT-Generated Code and StackOverflow Answers. In *IEEE Security and Privacy Workshops (SPW)*. 2024.

[21] Adriano Torres, Pedro Costa, Luis Amaral, Jonata Pastro, Rodrigo Bonifácio, Owolabi Legunsen, **M. d'Amorim**, Eric Bodden, and Edna Dias Canedo. Runtime Verification of Crypto APIs: An Empirical Study. In *IEEE Transactions on Software Engineering (TSE)*. 2023.

[22] Brittany Reid, Keila Barbosa, **M. d'Amorim**, Markus Wagner, and Christoph Treude. NCQ: code reuse support for Node.js developers. In *IEEE Transactions on Software Engineering (TSE)*. 2023.

[23] Lucas Cabral, Breno Miranda, Igor Lima, and **M. d'Amorim**. RVPrio: a Tool for Prioritizing Runtime Verification Violations. In *Software Testing, Verification and Reliability (STVR)*. 2022.

[24] Keila Barbosa, Ronivaldo Ferreira, Gustavo Pinto, **M. d'Amorim**, and Breno Miranda. Test Flakiness Across Programming Languages. In *IEEE Transactions on Software Engineering (TSE)*. 2022.

[25] Andrea Stocco, Paulo J. Nunes, **M. d'Amorim**, and Paolo Tonella. ThirdEye: Attention Maps for Safe Autonomous Driving Systems. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. October 2022.

[26] Sofia Reis, Rui Abreu, **M. d'Amorim**, and Daniel Fortunato. Leveraging Practitioners' Feedback to Improve a Security Linter. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. October 2022.

[27] Facundo Molina, Nazareno Aguirre, and **M. d'Amorim**. Fuzzing Class Specifications. In *International Conference on Software Engineering (ICSE)*. May 2022.

[28] Lucas Alcantara, Guilherme Padilha, Rui Abreu, and **M. d'Amorim**. Syrius: Synthesis of Rules for Intrusion Detectors. In *IEEE Transactions on Reliability (TRel)*. March 2021.

[29] Shouvick Mondal, Denini Silva, and **M. d'Amorim**. Soundy Automated Parallelization of Test Execution. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*. September 2021.

[30] Daniela Costa, Carlos Mello, and **M. d'Amorim**. A Comparative Study on Methods and Tools for Handwritten Mathematical Expression Recognition. In *ACM Symposium on Document Engineering (DocEng)*. August 2021.

[31] Jordan Henkel, Denini Silva, Leopoldo Teixeira, **M. d'Amorim**, and Thomas Reps. Shipwright: A Human-in-the-Loop System for Dockerfile Repair. In *International Conference on Software Engineering (ICSE)*. May 2021.

[32] Leopoldo Teixeira, Breno Miranda, Henrique Rebêlo, and **M. d'Amorim**. Demystifying the Challenges to Formally Specifying API Properties for Runtime Verification. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. April 2021.

[33] Marcello Cordeiro, Denini Silva, Leopoldo Teixeira, Breno Miranda, and **M. d'Amorim**. Shaker: a Tool for Detecting More Flaky Tests Faster. In *IEEE/ACM International Conference on Automated Software Engineering (ASE Tool Demonstrations)*. November 2021.

[34] Igor Lima, Jefferson Silva, Breno Miranda, Gustavo Pinto, and **M. d'Amorim**. Exposing Bugs in JavaScript Engines through Test Transplantation and Differential Testing. In *Journal of Systems and Software (JSS)*. November 2020.

[35] Rohit Gheyi, Márcio Ribeiro, Beatriz Sousa, Marcio A. Guimarães, Leo Fernandes, **M. d'Amorim**, Vander Alves, Leopoldo Teixeira, and Baldoino Fonseca. Identifying Method-Level Mutation Subsumption Relations using Z3. In *Information and Software Technology (IST)*. November 2020.

[36] Igor Lima, Jeanderson Candido, and **M. d'Amorim**. Practical Detection of CMS Plugin Conflicts in Large Plugin Sets. In *Information and Software Technology (IST)*. February 2020.

[37] Breno Miranda, Igor Lima, Owolabi Legunsen, and **M. d'Amorim**. Prioritizing Runtime Verification Violations. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 297-308, October 2020.

[38] Marcio A. Guimarães, Leo Fernandes, Márcio Ribeiro, **M. d'Amorim**, and Rohit Gheyi. Optimizing Mutation Testing by Discovering Dynamic Mutant Subsumption Relations. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 198–208, October 2020.

[39] Denini Silva, Leopoldo Teixeira, and **M. d'Amorim**. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *International Conference on Software Maintenance and Evolution (ICSME)*, pages 492–502, September 2020.

[40] Gustavo Pinto, Breno Miranda, Supun Dissanayake, **M. d'Amorim**, Christoph Treude, and Antonia Bertolino. What is the Vocabulary of Flaky Tests? In *International Conference on Mining Software Repositories (MSR)*, pages 492–502, July 2020.

[41] **M. d'Amorim**, Rui Abreu, and Carlos Mello. Visual Sketching: From Image Sketches to Code. In *International Conference on Software Engineering New Ideas and Emerging Results (ICSE NIER)*, pages 101–104, July 2020.

[42] Luis Melo, Igor Wiese, and **M. d'Amorim**. Using Docker to Assist Q&A Forum Users. In *IEEE Transactions on Software Engineering (TSE)*. December 2019.

[43] Sofia Reis, Rui Abreu, and **M. d'Amorim**. A Study of Demystifying the Combination of Dynamic Slicing and Spectrum-based Fault Localization. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4760-4766, August 2019.

[44] Davino Mauro Junior, Luis Melo, Harvey Lu, **M. d'Amorim**, and Atul Prakash. A Study of Vulnerability Analysis of Popular Smart Devices Through Their Companion Apps. In *IEEE Workshop on the Internet of Safe Things (SafeThings)*, May 2019.

[45] Xiangyu Li, **M. d'Amorim**, and Alessandro Orso. Intent-Preserving Test Repair. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 141-152, April 2019.

[46] Sabrina Souto and **M. d'Amorim**. Time-Space Efficient Regression Testing for Configurable Systems. In *Journal of Systems and Software (JSS)*, Volume 137, pages 733-746, 2018.

[47] Mattia Fazzini, Martin Prammer, **M. d'Amorim**, and Alessandro Orso. Automatically Translating Bug Reports into Test Cases for Mobile Apps. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 141-152, July 2018.

[48] Xiangyu Li, Shaowei Zhu, **M. d'Amorim**, and Alessandro Orso. Enlightened Debugging. In *International Conference on Software Engineering (ICSE)*, pages 82-92, May 2018.

[49] Jeanderson Candido, Luis Melo, and **M. d'Amorim**. Test Suite Parallelization in Open-Source Projects: a Study on its Usage and Impact. In *IEEE/ACM Intl. Conference on Automated Software Engineering (ASE)*, pages 838-848, November 2017.

[50] Sabrina Souto, **M. d'Amorim**, and Rohit Gheyi. Balancing Soundness and Efficiency for Practical Testing of Configurable Systems. In *International Conference on Software Engineering (ICSE)*, pages 632-642, May 2017.

[51] Alexandre Perez, Rui Abreu, and **M. d'Amorim**. Prevalence of Single-Fault Fixes and its Impact on Fault Localization. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 12-22, March 2017.

[52] Xiangyu Li, **M. d'Amorim**, and Alessandro Orso. Iterative User-Driven Fault Localization. In *Haifa Verification Conference (HVC)*, pages 82-98, November 2016.

[53] Paulo Barros, René Just, Suzanne Millstein, Paul Vines, Werner Dietl, **M. d'Amorim**, and Michael D. Ernst. Static Analysis of Implicit Control Flow: Resolving Java Reflection and Android Intents. In *IEEE/ACM Intl. Conference on Automated Software Engineering (ASE)*, pages 669-679, November 2015.

[54] Mateus Borges, Antonio Filieri, **M. d'Amorim**, and Corina S. Păsăreanu. Iterative Distribution-Aware Sampling for Probabilistic Software Analysis. In *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 866-877, Aug-Sept 2015.

[55] Sabrina Souto, Divya Gopinath, **M. d'Amorim**, Darko Marinov, Sarfraz Khurshid, and Don Batory. Faster Bug Detection for Software Product Lines with Incomplete Feature Models. In *International Systems and Software Product Line Conference (SPLC)*, pages 151–160, July 2015.

[56] Andrei Rimsa, **M. d'Amorim**, Fernando M. Q. Pereira, and Roberto S. Bigonha. Efficient Static Checker for Tainted Variable Attacks. In *Science of Computer Programming (SCP)*, Volume 80, pages 91–105, February 2014.

[57] Tianhai Liu, Mateus Borges, **M. d'Amorim**, and Mana Taghidiri. A Comparative Study of Incremental Constraint Solving Approaches in Symbolic Execution. In *Haifa Verification Conference (HVC)*, pages 284-299, November 2014.

[58] Quoc-Sang Phan, Pasquale Malacaria, Corina S. Păsăreanu, and **M. d'Amorim**. Quantifying Information Leaks using Reliability Analysis. In *International SPIN Symposium on Software Model Checking*, pages 105-108, July 2014.

[59] Mateus Borges, Antonio Filieri, **M. d'Amorim**, Corina S. Păsăreanu, and Willem Visser. Compositional Solution Space Quantification for Probabilistic Software Analysis. In *ACM/SIGPLAN Programming Language Design and Implementation (PLDI)*, pages 123-132, June 2014.

[60] José Carlos de Campos, Rui Abreu, Gordon Fraser, and **M. d'Amorim**. Entropy-based Test Generation for Improved Fault Localization. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 257–267, November 2013.

[61] Chang Hwan Peter Kim, Darko Marinov, Sarfraz Khurshid, Don Batory, Sabrina Souto, Paulo Barros, and **M. d'Amorim**. SPLat: Lightweight Dynamic Analysis for Reducing Combinatorics in Testing Configurable Systems. In *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 257–267, August 2013.

[62] Mateus Borges, **M. d'Amorim**, Saswat Anand, David Bushnell, and Corina S. Păsăreanu. Symbolic Execution with Interval Solving and Meta-heuristic Search. In *IEEE International Conference on Software Testing, Verification, and Validation (ICST)*, pages 111-120, April 2012.

[63] Elton Alves, Milos Gligoric, Vilas Jagannath, and **M. d'Amorim**. Improved Lightweight Debugging with Dynamic Slicing and Change Data. In *IEEE/ACM International Symposium on Automated Software Engineering (ASE)*, pages 520-523, November 2011.

[64] Matheus Souza, Mateus Borges, **M. d'Amorim**, and Corina S. Păsăreanu. CORAL: Solving Complex Constraints in Symbolic PathFinder. In *NASA Formal Methods Symposium (NFM)*, pages 359-374, April 2011.

[65] Andrei Rimsa, **M. d'Amorim**, and Fernando M. Q. Pereira. Efficient Tainted Flow Analysis. In *ETAPS Intl. Conference on Compiler Construction (CC)*, pages 124-143, March 2011.

[66] A. Sobeih, **M. d'Amorim**, M. Viswanathan, Darko Marinov, and J. Hou. Assertion checking in J-Sim simulation models of network protocols. In *Transactions of The Society for Modeling and Simulation International (Simulation)*, Volume 86, Number 11, 651-673, November 2010.

[67] M. Takaki, D. Cavalcanti, Rohit Gheyi, J. Iyoda, **M. d'Amorim**, and R. Prudencio. Randomized Constraint Solvers: A comparative study. In *Innovations in Systems and Software Engineering: a NASA journal (ISSE)*, Volume 6, Number 3, 243-253, September 2010.

[68] Andrei Rimsa, **M. d'Amorim**, and Fernando M. Q. Pereira. Efficient Static Checker for Tainted Variable Attacks. In *Brazilian Symposium on Programming Languages (SBLP)*, September 2010.

[69] M. Takaki, D. Cavalcanti, Rohit Gheyi, J. Iyoda, **M. d'Amorim**, and R. Prudencio. A Comparative Study of Randomized Constraint Solvers for Random-Symbolic Testing. In *NASA Formal Methods Symposium (NFM)*, pages 56-65, April 2009.

[70] C. Bertolini, G. Peres, **M. d'Amorim**, and A. Mota. An Empirical Evaluation of Automated Black Box Testing Techniques for Crashing GUIs. In *IEEE International Conference on Software Testing, Verification, and Validation (ICST)*, pages 21-30, April 2009.

[71] **M. d'Amorim**, S. Lauterburg, and Darko Marinov. Delta Execution for Efficient State-Space Exploration. In *IEEE Transactions on Software Engineering (TSE)*, Vol. 34, No. 5, pages 597-613, October 2008.

[72] T. Gvero, Milos Gligoric, S. Lauterburg, **M. d'Amorim**, Darko Marinov, and Sarfraz Khurshid. State Extensions for Java PathFinder. In *International Conference on Software Engineering (ICSE Demo)*, pages 863-866, May 2008.

[73] **M. d'Amorim**, S. Lauterburg, and Darko Marinov. Delta Execution for Efficient State-Space Exploration of Object-Oriented Programs. In *ACM/SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 50-60, July 2007.

[74] Y. Zhou, Darko Marinov, W. Sanders, C. Zilles, **M. d'Amorim**, S. Lauterburg, R. Lefever, and J. Tucek. Delta Execution for Software Reliability. In *Workshop on Hot Topics in System Dependability (HotDep)*. June 2007.

[75] **M. d'Amorim**, A. Sobeih, and Darko Marinov. Optimized execution of deterministic blocks in Java PathFinder. In *International Conference on Formal Engineering Methods (ICFEM)*, pages 549-567, November 2006.

[76] **M. d'Amorim**, C. Pacheco, T. Xie, Darko Marinov, and Michael D. Ernst. An empirical comparison of automated generation and classification techniques for object-oriented unit testing. In *IEEE/ACM International Symposium on Automated Software Engineering (ASE)*, pages 59-68, September 2006.

[77] **M. d'Amorim** and G. Roşu. An Equational Specification for the Scheme Language. In *Journal of Universal Computer Science (JUCS)*, 11(7), pages 1327-1348, July 2005.

[78] **M. d'Amorim** and G. Roşu. Efficient Monitoring of ω-Languages. In *Intl. Conf. on Computer Aided Verification (CAV)*, pages 364-378, July 2005.

[79] **M. d'Amorim** and G. Roşu. An Equational Specification for the Scheme Language. In *Simpósio Brasileiro de Linguagens de Programação (SBLP)*, pages 229-242, June 2005.

[80] F. Chen, **M. d'Amorim**, and G. Roşu. Checking and Correcting Behaviors of Java Programs at Runtime with Java-MOP. In *5th Workshop on Runtime Verification (RV)*, pages 3-20, July 2005.

[81] **M. d'Amorim** and K. Havelund. Event-Based Runtime Verification of Java Programs. In *ACM/SIGSOFT International Workshop on Dynamic Analysis (WODA)*, pages 15-21, July 2005.

[82] F. Chen, **M. d'Amorim**, and G. Roşu. A Formal Monitoring-based Framework for Software Development and Analysis. In *International Conference on Formal Engineering Methods (ICFEM)*, pages 357-372, November 2004.

[83] **M. d'Amorim**, C. Nogueira, G. Santos, A. Souza, and P. Borba. Integrating Code Generation and Refactoring. In *Workshop on Generative Programming* (ECOOP event), June 2002.

[84] **M. d'Amorim** and C. Ferraz. Designing Jini Distributed Services: A Framework to support the development of reliable component networks. In *Workshop on Language Mechanisms for Programming Software Components* (OOPSLA event), October 2001.